

ESTRUTURAS DE REPETIÇÃO

Automatizando tarefas com Python

Nesta aula, você aprenderá a usar laços de repetição para criar programas mais dinâmicos, eficientes e capazes de executar tarefas repetitivas automaticamente.

****Responsável:** Raul Nascimento Belém Pontes**

POR QUE USAR REPETIÇÃO?

Estruturas de repetição permitem executar um mesmo bloco de código várias vezes, sem precisar escrever a mesma instrução repetidamente.

1. EVITA REPETIÇÃO

Reduz código duplicado e deixa o programa mais limpo.

2. AUTOMATIZA TAREFAS

Executa ações em sequência, como percorrer listas, contar valores e repetir processos.

3. GANHA EFICIÊNCIA

Facilita a resolução de problemas com lógica iterativa.

Em vez de escrever 100 comandos print, usamos um laço para repetir a ação automaticamente.

LAÇO FOR

O for é utilizado quando sabemos a quantidade de vezes que queremos repetir uma ação ou quando queremos percorrer uma sequência de dados.

```
for numero in range(1, 6):  
    print(numero)
```

O que acontece?

- A variável numero recebe valores de 1 até 5.
- O bloco indentado é executado a cada repetição.
- A função range ajuda a criar uma sequência numérica.

Uso comum: percorrer listas, repetir cálculos, exibir sequências e processar coleções de dados.

LAÇO WHILE

O while repete um bloco de código enquanto uma condição for verdadeira. Ele é muito útil quando não sabemos exatamente quantas repetições serão necessárias.

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

ATENÇÃO

No while, é fundamental atualizar a condição dentro do laço para evitar um loop infinito.

Destaque: Condição de parada

CONTADORES E ACUMULADORES

Dentro dos laços, dois recursos aparecem com frequência: contadores e acumuladores. Eles ajudam a controlar repetições e guardar resultados.

TIPO	EXEMPLO	USO
Contador	<code>contador = contador + 1</code>	Controla a quantidade de repetições.
Acumulador	<code>total = total + valor</code>	Soma ou acumula valores ao longo do laço.
Atualização curta	<code>contador += 1</code>	Forma resumida de atualizar uma variável.

Ideia central: A cada repetição, uma variável pode ser atualizada para contar, somar ou controlar o fluxo.

CONDIÇÕES DE PARADA

A condição de parada define quando o laço deve terminar. Sem ela, o programa pode repetir para sempre, gerando um loop infinito.

1

INÍCIO

Define variáveis

2

CONDIÇÃO

Verifica se continua

3

REPETIÇÃO

Executa o bloco

4

ATUALIZAÇÃO

Aproxima do fim

Exemplo de código:

```
while senha != "1234":  
    senha = input("Digite a senha: ")
```

EXEMPLO PRÁTICO

Agora vamos usar repetição, contador e acumulador para somar os números de 1 até 5.

```
total = 0

for numero in range(1, 6):
    total = total + numero

print("Soma final:", total)
```

RESULTADO NO CONSOLE:

Soma final: 15

O acumulador **total** começa em zero e recebe a soma de cada número a cada repetição do laço for.

Esse padrão é extremamente comum em cálculos de médias, totais de vendas, relatórios financeiros e análises de grandes volumes de dados.

RESUMO DA AULA

- ✓ Estruturas de repetição evitam código duplicado e tornam o programa mais limpo.
- ✓ O laço **for** é ideal para percorrer sequências e repetir ações com quantidade definida.
- ✓ O laço **while** executa um bloco de código enquanto uma condição for verdadeira.
- ✓ Contadores, acumuladores e condições de parada são essenciais para controlar o fluxo dos laços.
- ✓ Repetições permitem resolver problemas complexos através de lógica iterativa eficiente.

PRÓXIMA AULA

Estruturas de Dados

Te vejo lá!